Hierarchical RL Trading Framework — Technical Spec

Part 1: State / Action / Reward Specification

1.1 High-Level Policy (Strategy Selection & Capital Allocation)

Time scale: Acts every N bars (e.g., every 4 hours or daily), or on regime-change triggers.

State space (S_high):

Feature group	Examples
Regime indicators	HMM/change-point state probabilities, realized volatility (20/60-day), ADX trend strength
Macro/cross-asset	VIX level & term structure, yield curve slope, sector correlation matrix summary (e.g., avg pairwise correlation)
Portfolio state	Current allocation vector, unrealized PnL, current drawdown, days since last rebalance
Momentum/mean-reversion signals	Rolling Sharpe of each candidate strategy over trailing window

Represent as a fixed-length vector or embed the correlation matrix/time series with a small encoder (GRU/1D-CNN) if you want to learn representations end-to-end.

Action space (A_high): Discrete or hybrid

- Discrete: choose among K strategy "options" (e.g., trend-following, mean-reversion, defensive/cash, breakout) — this is the Options-framework formulation
- Continuous add-on: target portfolio-level exposure/leverage scalar $\in [0, 1]$ or $[-1, 1]$ if shorting allowed
- Optionally: target volatility budget the low-level agent must respect

Reward (R_high): Computed over the high-level decision window (not per-tick)

- Differential Sharpe ratio (Moody & Saffell) — good for online RL since it's incremental
- Or: `R_high = return_over_window - λ·drawdown_penalty - κ·switching_cost`
- Switching cost matters — penalize option changes to avoid thrashing between strategies every step

Termination condition (if using Options framework): Learn a termination probability $\beta(s)$ conditioned on regime-change confidence — this is the "regime-conditioned termination" novelty angle from before. Simple version: terminate if regime classifier confidence drops below threshold or fixed max duration reached.

1.2 Low-Level Policy (Trade Execution)

Time scale: Acts every bar/tick within the window set by the high-level policy.

State space (S_low):

Feature group	Examples
Market microstructure	Bid-ask spread, order book imbalance (if available), recent volume, short-term volatility
Price action	OHLCV over short lookback, VWAP deviation
Execution context	Remaining target position to fill, time remaining in execution window, current inventory
Inherited context	The high-level action/option currently active (passed down as conditioning input)

Action space (A_low): Continuous (SAC/DDPG/TD3 suited)

- Order size as fraction of remaining target (0 to 1)
- Optionally: order type/aggressiveness (market vs. limit offset) if you have order-book-level data
- Optionally: pause/wait action to model timing choice

Reward (R_low): Per-step, execution-quality focused

- `R_low = -slippage - transaction_cost - λ·inventory_risk_penalty`
- Slippage measured vs. arrival price or VWAP benchmark (implementation shortfall)
- If no order-book data available, approximate transaction cost with a fixed bps + volume-based market impact model (e.g., square-root impact law)

Key design point: the low-level reward should NOT directly reward PnL/direction — that's the high-level's job. Keeping the objectives separated is what makes the hierarchy meaningful rather than cosmetic.

1.3 Interfaces Between Levels

- High-level output → passed to low-level as: (a) conditioning input in state, and (b) target position/exposure to achieve
- Low-level reports back → realized execution cost feeds into high-level's effective return calculation (so high-level indirectly "feels" bad execution)
- Suggested training order: pretrain/warm-start low-level execution agent on a simpler benchmark task (e.g., minimize slippage for random target trades) before joint training, to avoid a moving-target problem where both levels are randomly initialized simultaneously

---

Part 2: Evaluation & Experiment Section (Draft)

2.1 Research Questions

1. Does hierarchical decomposition improve risk-adjusted returns over a flat (single-level) DRL baseline?
2. Does regime-conditioned strategy switching outperform fixed-interval switching?
3. How much of the improvement comes from better execution vs. better strategy selection? (ablation)
4. How robust is performance across distinct market regimes (bull/bear/high-vol/sideways)?

2.2 Baselines

- Buy-and-hold
- Classical rule-based strategy (e.g., moving-average crossover, or risk-parity rebalancing)
- Flat single-level DRL agent (PPO or SAC directly on raw state → trade action), same feature set
- Hierarchical framework without regime conditioning (fixed-interval option switching)
- Full proposed system

2.3 Metrics

- **Return-based:** cumulative return, annualized return, CAGR
- **Risk-adjusted:** Sharpe, Sortino, Calmar ratio
- **Risk/tail:** max drawdown, drawdown duration, CVaR (95%/99%)

- **Execution quality:** average slippage vs. benchmark, turnover, transaction cost as % of PnL
- **Statistical robustness:** bootstrap confidence intervals on Sharpe ratio (e.g., 1,000 resamples of return blocks), or the probabilistic Sharpe ratio (Bailey & López de Prado)

2.4 Experimental Protocol

- **Walk-forward validation** (see Part 3) rather than a single split
- Report results per fold AND aggregated, so variance across regimes is visible — don't just report one blended number
- **Ablation table:** hierarchical vs. flat; with/without regime conditioning; with/without risk-exposure module — isolate which component drives the gains
- **Sensitivity analysis:** vary transaction cost assumptions (e.g., 5bps, 10bps, 20bps) to show results aren't an artifact of an optimistic cost model
- **Statistical significance:** compare Sharpe ratios between models using a paired test on the walk-forward fold returns (e.g., Jobson-Korkie test with Memmel correction), not just eyeballing the numbers

2.5 Reporting Structure (suggested for your capstone writeup)

1. In-sample training curves (reward convergence per level)
2. Out-of-sample equity curves for all baselines, overlaid
3. Metrics table (rows = models, columns = metrics) split by regime period
4. Ablation table
5. Discussion of failure cases (e.g., a regime the model handled badly, and why)

Part 3: Dataset Selection & Walk-Forward Backtesting Pipeline

3.1 Dataset Recommendations

Use case	Source	Notes
Equities (daily/intraday)	Yahoo Finance (`yfinance`), Stooq, or Alpha Vantage (free tier)	Good for capstone scope; daily bars easiest to start
Equities (minute-level, for execution realism)	Polygon.io, Alpaca Markets (free historical data with account)	Needed if you want real microstructure features

Use case	Source	Notes
Crypto (24/7, high volatility, easy access)	Binance public API, CCXT library	Popular in RL-trading papers because data is free and abundant
FX	Dukascopy historical data (free)	Good liquidity, continuous market
Regime-labeled macro data	FRED (St. Louis Fed) for VIX, yield curve, rates	Pair with your price data for context features

Recommendation for a capstone: Start with a diversified equity or ETF universe (e.g., S&P 500 sector ETFs, or 10–20 liquid large-cap names) at daily bars for the high-level strategy layer, and pull minute bars for a subset to build the execution layer realistically. This keeps compute manageable while still supporting the hierarchy.

Regime periods to include (US equities example):

- Bull: 2016–2019
- COVID crash: Feb–Apr 2020
- Recovery/melt-up: mid-2020–2021
- Rate-hike bear market: 2022
- Sideways/choppy: pick a range-bound period specific to your asset

3.2 Walk-Forward Pipeline Design

```
[Full dataset: 2015 --------------------------- 2024]

Fold 1: Train [2015-2017] -> Validate [2018] -> Test [2018 H2]
Fold 2: Train [2015-2018] -> Validate [2019] -> Test [2019 H2]
Fold 3: Train [2015-2019] -> Validate [2020 H1] -> Test [2020 H2]  <-
includes COVID
Fold 4: Train [2015-2021] -> Validate [2022 H1] -> Test [2022 H2] <- includes
rate-hike bear
...
```

Either **expanding window** (train set grows each fold, more realistic for "deploy and keep learning") or **rolling window** (fixed-size train set, tests adaptability to regime drift, arguably better for your non-stationarity claim). Rolling window is more defensible for your "adaptive to non-stationary markets" thesis — consider using it as primary and expanding window as a robustness check.

3.3 Implementation Sketch (Python)

python

```python
import pandas as pd
import numpy as np
from dataclasses import dataclass

@dataclass
class Fold:
    train: pd.DataFrame
    val: pd.DataFrame
    test: pd.DataFrame
    fold_id: int

def make_walk_forward_folds(df, train_years=3, val_months=6, test_months=6, ste
    """
    df: DataFrame indexed by date, sorted ascending.
    Produces overlapping (rolling) or expanding walk-forward folds.
    """
    folds = []
    start = df.index.min()
    end = df.index.max()
    fold_id = 0
    cursor = start + pd.DateOffset(years=train_years)

    while True:
        train_start = start if not rolling else cursor - pd.DateOffset(years=tr
        train_end = cursor
        val_end = train_end + pd.DateOffset(months=val_months)
        test_end = val_end + pd.DateOffset(months=test_months)

        if test_end > end:
            break

        train = df.loc[train_start:train_end]
        val = df.loc[train_end:val_end]
        test = df.loc[val_end:test_end]

        folds.append(Fold(train, val, test, fold_id))
        fold_id += 1
        cursor += pd.DateOffset(months=step_months)

    return folds

# Usage:
# prices = pd.read_csv("your_ohlcv.csv", index_col="date", parse_dates=True)
```

```
# folds = make_walk_forward_folds(prices, train_years=3, rolling=True)
# for fold in folds:
#     train_high_level_agent(fold.train)
#     tune_hyperparams(fold.val)
#     equity_curve = run_backtest(fold.test)
#     record_metrics(equity_curve, fold.fold_id)
```

Pipeline checklist:

1. Feature engineering must use only rolling/expanding windows computed **within** each fold's train period — never fit normalization (mean/std, min/max) on the full dataset before splitting (this is the #1 look-ahead bias bug in student RL-trading projects)
2. Retrain (or fine-tune) both hierarchy levels at the start of each fold on that fold's train set
3. Freeze the policy during val/test — no further gradient updates
4. Aggregate metrics across folds using both mean and standard deviation — high variance across folds is itself a finding worth reporting
5. Store per-fold equity curves so you can plot them separately (useful for showing which regime the model struggles in)

3.4 Cost Model (needed since you likely won't have real order-book data)

python

```python
def transaction_cost(trade_value, spread_bps=5, impact_coef=0.1, avg_daily_volu
    fixed_cost = trade_value * (spread_bps / 10000)
    market_impact = 0
    if avg_daily_volume and trade_size:
        participation_rate = trade_size / avg_daily_volume
        market_impact = trade_value * impact_coef * np.sqrt(participation_rate)
    return fixed_cost + market_impact
```

Use this inside the low-level reward function so R_low reflects realistic frictions even without tick-level data.

Suggested Next Steps

1. Lock in your dataset and universe (recommend starting with 10–20 liquid ETFs/stocks, daily bars, extend to minute bars later if time allows)
2. Implement the walk-forward fold generator and verify no look-ahead leakage

3. Build the flat DRL baseline first (single PPO/SAC agent) — this becomes both a sanity check and Table 1 comparison
4. Then build the low-level execution agent in isolation against the cost model, validate it reduces slippage vs. naive execution
5. Combine into the full hierarchy last, once both pieces work independently